

Nama : Nyssa Leilani Calista

Nim : 25091397062

Kelas : 2025B

PENJELASAN DARI PROGRAM TERSEBUT

```
import argparse
```

```
import random
```

```
import numpy as np
```

```
import matplotlib
```

```
matplotlib.use("TkAgg") # CMD Windows GUI backend
```

```
import matplotlib.pyplot as plt
```

```
from matplotlib.animation import FuncAnimation
```

```
def plan_inserts(keys, size=20):
```

```
    table = [None] * size
```

```
    frame_data = []
```

```
    load_factors = []
```

```
    for step, key in enumerate(keys, 1):
```

```
        start = hash(key) % size
```

```
        idx = start
```

```
        probes = 0
```

```
    def snap(phase, placed=False):
```

```
frame_data.append({  
    "phase": phase,  
    "step": step,  
    "key": key,  
    "start": start,  
    "idx": idx,  
    "probes": probes,  
    "table": list(table),  
    "placed": placed  
})
```

```
snap("start", placed=False)
```

```
while table[idx] is not None:
```

```
    snap("collision", placed=False)
```

```
    probes += 1
```

```
    idx = (idx + 1) % size
```

```
    snap("probe", placed=False)
```

```
table[idx] = key
```

```
snap("place", placed=True)
```

```
lf = sum(v is not None for v in table) / size
```

```
load_factors.append(lf)
```

```
snap("pause", placed=True)
```

```
return frame_data, load_factors
```

```
def main():
```

```
    parser = argparse.ArgumentParser()
```

```
    parser.add_argument("--size", type=int, default=20)
```

```
    parser.add_argument("--nkeys", type=int, default=18)
```

```
    parser.add_argument("--seed", type=int, default=42)
```

```
    parser.add_argument("--save", choices=["none", "gif"], default="none")
```

```
    args = parser.parse_args()
```

```
    random.seed(args.seed)
```

```
    keys = [f'k{i}' for i in range(args.nkeys)]
```

```
    random.shuffle(keys)
```

```
    frames, load_factors = plan_inserts(keys, size=args.size)
```

```
    # ===== FIGURE LAYOUT =====
```

```
    fig = plt.figure(figsize=(12, 6), constrained_layout=True)
```

```
    gs = fig.add_gridspec(2, 1, height_ratios=[3, 1])
```

```
    ax_table = fig.add_subplot(gs[0])
```

```
    ax_lf = fig.add_subplot(gs[1])
```

```
    fig.suptitle("Hash Table Premium+ (Linear Probing) — Keyboard Control", fontsize=14)
```

```
    # ---- TABLE GRID ----
```

```

n = args.size
ax_table.set_xlim(-0.5, n - 0.5)
ax_table.set_ylim(-0.5, 0.5)
ax_table.set_yticks([])
ax_table.set_xticks(range(n))
ax_table.set_xlabel("Index Bucket")

rects, texts = [], []
for i in range(n):
    r = plt.Rectangle((i - 0.5, -0.35), 1.0, 0.7, fill=True, alpha=0.25)
    ax_table.add_patch(r)
    rects.append(r)
    t = ax_table.text(i, 0, "", ha="center", va="center", fontsize=9)
    texts.append(t)

info = ax_table.text(0.01, 1.08, "", transform=ax_table.transAxes, fontsize=11)
legend = ax_table.text(
    0.01, 1.01,
    "SPACE: pause/resume | ←/→: step (pause) | R: restart | Q/Esc: quit",
    transform=ax_table.transAxes, fontsize=9
)

# ---- LOAD FACTOR CHART ----
ax_lf.set_xlim(1, max(2, args.nkeys))
ax_lf.set_ylim(0, 1.05)
ax_lf.set_xlabel("Insert ke-")
ax_lf.set_ylabel("Load Factor")

```

```
ax_lf.grid(True, alpha=0.25)
```

```
lf_line, = ax_lf.plot([], [])
```

```
lf_dot, = ax_lf.plot([], [], marker="o", linestyle="")
```

```
bar_bg = plt.Rectangle((0.02, 0.15), 0.96, 0.2, transform=ax_lf.transAxes, alpha=0.15)
```

```
bar_fg = plt.Rectangle((0.02, 0.15), 0.00, 0.2, transform=ax_lf.transAxes, alpha=0.35)
```

```
ax_lf.add_patch(bar_bg)
```

```
ax_lf.add_patch(bar_fg)
```

```
lf_text = ax_lf.text(0.02, 0.45, "", transform=ax_lf.transAxes, fontsize=10)
```

```
# ===== PLAYER STATE =====
```

```
state = {
```

```
    "i": 0,      # current frame index
```

```
    "paused": False # paused flag
```

```
}
```

```
def render(frame):
```

```
    """Render a single frame dict."""
```

```
    table = frame["table"]
```

```
    start = frame["start"]
```

```
    idx = frame["idx"]
```

```
    phase = frame["phase"]
```

```
    # reset buckets
```

```
    for j in range(n):
```

```
        rects[j].set_alpha(0.25)
```

```

    rects[j].set_linewidth(1.0)

    texts[j].set_text("" if table[j] is None else str(table[j]))

# highlight target hash bucket
rects[start].set_alpha(0.55)
rects[start].set_linewidth(2.5)

# highlight current idx
rects[idx].set_alpha(0.75)
rects[idx].set_linewidth(3.0)

# collision emphasis
if phase == "collision":
    rects[idx].set_alpha(0.9)

filled = sum(v is not None for v in table)
lf = filled / n

info.set_text(
    f'Frame    {state['i']+1}/{len(frames)}    |    Step    {frame['step']}/{args.nkeys}    |
key='{frame['key']}' | "
    f'hash%size={start} | idx={idx} | probes={frame['probes']} | load_factor={lf:.2f} |
phase={phase} | "
    f'{'PAUSED' if state['paused'] else 'PLAY'}'"
)

# completed inserts so far
completed = frame["step"] - 1 + (1 if frame["placed"] else 0)

```

```

if completed <= 0:
    x, y = [], []
else:
    x = list(range(1, completed + 1))
    y = load_factors[:completed]

lf_line.set_data(x, y)

if x:
    lf_dot.set_data([x[-1]], [y[-1]])
    bar_fg.set_width(0.96 * y[-1])
    lf_text.set_text(f"Load Factor: {y[-1]:.2f} ({int(y[-1]*100)}%)")
else:
    lf_dot.set_data([], [])
    bar_fg.set_width(0.00)
    lf_text.set_text("Load Factor: 0.00 (0%)")

return rects + texts + [info, legend, lf_line, lf_dot, bar_fg, lf_text]

def update(_):
    """Animation tick: advance if not paused; otherwise keep frame."""
    if not state["paused"]:
        state["i"] = min(state["i"] + 1, len(frames) - 1)
    return render(frames[state["i"]])

# Init render
def init():
    return render(frames[state["i"]])

```

```
ani = FuncAnimation(  
    fig, update, init_func=init,  
    frames=np.arange(len(frames)), # timer ticks, actual frame controlled by state["i"]  
    interval=450, blit=False, repeat=False  
)
```

```
def on_key(event):  
    k = event.key  
    if k == " ":  
        state["paused"] = not state["paused"]  
        fig.canvas.draw_idle()  
  
    elif k == "right":  
        if state["paused"]:  
            state["i"] = min(state["i"] + 1, len(frames) - 1)  
            render(frames[state["i"]])  
            fig.canvas.draw_idle()  
  
    elif k == "left":  
        if state["paused"]:  
            state["i"] = max(state["i"] - 1, 0)  
            render(frames[state["i"]])  
            fig.canvas.draw_idle()  
  
    elif k in ("r", "R"):  
        state["i"] = 0
```

```

        state["paused"] = True # restart dalam kondisi pause biar enak step
        render(frames[state["i"]])
        fig.canvas.draw_idle()

    elif k in ("escape", "q", "Q"):
        plt.close(fig)

fig.canvas.mpl_connect("key_press_event", on_key)

if args.save == "gif":
    try:
        from matplotlib.animation import PillowWriter
        ani.save("hash_table_premium_keyboard.gif", writer=PillowWriter(fps=2))
        print("Tersimpan: hash_table_premium_keyboard.gif")
    except Exception as e:
        print("Gagal simpan GIF. Pastikan 'pillow' terpasang. Error:", e)

plt.show()

if __name__ == "__main__":
    main()

```

Ini adalah penjelasan konsep **Hash Table** dan **Linear Probing** dari kode tersebut menggunakan perumpamaan "**Sistem Parkir Otomatis**" yang sangat mudah dipahami.

Analogi Sederhana: Sistem Parkir Mobil Otomatis

Bayangkan sebuah gedung parkir dengan **20 slot (kotak parkir)** yang diberi nomor 0 sampai 19. Kode Python tersebut mensimulasikan bagaimana mobil-mobil masuk ke parkiran ini menggunakan aturan tertentu.

1. Apa itu Hash Table? (Gedung Parkir)

- **Gedung Parkir:** Adalah *Hash Table*-nya.
- **Nomor Plat Mobil:** Adalah *Key* (Data yang ingin disimpan).
- **Petugas Parkir (Fungsi Hash):** Saat mobil datang, petugas menghitung nomor plat mobil untuk menentukan di slot mana mobil itu **seharusnya** parkir.
 - *Rumus:* Nomor Plat % 20. Jika hasilnya 5, mobil harus ke kotak nomor 5.

2. Apa itu Linear Probing? (Mencari Kotak Kosong)

Masalah muncul jika kotak nomor 5 sudah terisi mobil lain (**Tabrakan/Collision**).

- Dalam kode ini, aturannya simpel: Jika kotak tujuan penuh, mobil akan maju ke **kotak berikutnya tepat di depannya** (nomor 6).
- Jika nomor 6 penuh juga? Maju lagi ke nomor 7. Begitu seterusnya sampai menemukan kotak kosong. Inilah yang disebut **Linear Probing**.

Penjelasan Alur Kode (Step-by-Step)

1. Perencanaan (Function `plan_inserts`):

Sebelum animasi dimulai, kode membuat "skenario" perjalanan setiap mobil. Ia mencatat kapan mobil menabrak, kapan ia harus maju ke slot sebelah, dan di mana akhirnya ia parkir.

2. Visualisasi Tabel (Kotak Atas):

- **Kotak Biru Muda:** Slot parkir kosong.
- **Kotak Tebal/Terang:** Menunjukkan posisi petugas saat sedang mengecek slot tersebut.
- **Teks di dalam kotak:** Nama mobil (contoh: k1, k2) yang sudah berhasil parkir.

3. Load Factor (Grafik Bawah):

Ini adalah indikator "Kepadatan Parkiran".

- Jika parkiran hampir penuh (misal 18 dari 20 slot terisi), *Load Factor*-nya 0.90 (90%).
- Semakin tinggi angka ini, semakin sulit mobil baru menemukan tempat parkir yang kosong, dan sistem akan semakin lambat karena harus terus-menerus "maju satu langkah" mencari slot.

Kontrol Navigasi (Interaksi)

Dalam aplikasi tersebut, Anda bisa mengendalikan waktu layaknya menonton video:

- **SPACE (Spasi):** Untuk mengerem (pause) atau menjalankan simulasi parkir.
- **PANAH KANAN/KIRI:** Untuk melihat proses parkir mobil satu per satu secara perlahan (Frame-by-Frame).
- **HURUF R:** Mengulang seluruh proses parkir dari awal (mobil pertama).

Kesimpulan:

Kode ini mendemonstrasikan bahwa semakin penuh sebuah penyimpanan data (*Hash Table*), semakin banyak "tabrakan" yang terjadi, dan komputer butuh waktu lebih lama untuk menempatkan data baru karena harus mencari celah kosong satu per satu secara berurutan.